

가이드 03 · v1.0

v1.0 PreCheck Log Analysis System

로그분석 스케줄 작성 가이드

Analyze Schedule Configuration Guide

analyze 서버가 어떤 서버의 수집 로그를, 언제, 어떤 방식으로 분석할지 정의하는 스케줄 설정 파일 작성 가이드입니다. collect 스케줄과 형식이 유사하지만 서버 IP·SFTP 개념이 없고 파일경로가 선택 항목이라는 차이가 있습니다.

파일 위치	~/cfg/PreCheck_AnalyzeLogs_Schedule.conf
파싱 주체	analyze/parser/AnalyzeScheduleParser.java
폴링 주체	analyze/scheduler/AnalyzeScheduler.java (60초 간격 폴링)

01 공통 포맷 — collect와 다른 점

COMMON FORMAT

[서버 ID][수집대상파일경로][스케줄표현식]

- 서버ID — 필수, collect 스케줄의 서버ID와 동일 값 사용
- 수집대상파일경로 — 선택(점선 토큰). 생략 시 3토큰 [서버ID][스케줄표현식] 형태
- 스케줄표현식 — collect와 동일 문법(배치|주기)

파일경로가 왜 선택인가

analyze는 파일을 직접 읽지 않고 collect가 이미 적재한 TB_COLLECT_LOG 를 서버ID 기준으로 조회합니다. 파일경로는 이력 구분/표시 용도로만 쓰이므로 생략 가능합니다(2토큰 또는 3토큰 모두 허용).

중복 키 — 마지막 정의가 우선

서버ID + 파일경로 조합이 파일 내에 중복되면, LinkedHashMap 의 remove→put 방식으로 마지막에 등장한 라인이 최종 적용됩니다(가이드4의 정책 파일과 동일한 규칙).

02 스케줄표현식 — 배치 vs 주기

SCHEDULE EXPRESSION BY MODE

배치

BATCH — 지정 요일 지정 시각에 하루 1회 분석

배치|요일|시작시각(HH:mm:ss)

동작	지정 요일의 시작시각이 되면 그 시점까지 쌓인 미분석 TB_COLLECT_LOG 를 한 번에 분석
적합한 경우	실시간성이 필요 없고 하루 한 번 마감성 분석이면 충분한 서버(예: 주문체결 마감 점검)

[dcodb01-주문체결][배치|1-5|180000]
→ 평일(1-5) 18:00:00에 1회 분석

주기

PERIODIC — 시작~종료 사이 간격(분)마다 증분 분석(기본 동작)

주기|요일|시작시각|간격(분)|종료시각

동작	매 간격마다 TB_ANALYZE_HISTORY.lastAnalyzeLogId 이후로 새로 쌓인 TB_COLLECT_LOG 만 분석(중복 분석 방지)
유효성	시작시각 < 종료시각이어야 함. 같거나 크면(예: 180001 10 090001) 파싱 실패로 라인 무시

[dlprem01-테스트개발][주기|*|000501|1|230501]
→ 매일 00:05:01부터 23:05:01까지 1분 간격으로 증분 분석

03 스케줄러 실행 판정

SCHEDULER EXECUTION LOGIC

항목	동작
폴링 주기	<code>@Scheduled(fixedDelay=60000)</code> — 60초마다 스케줄 목록 검사(collect의 1초보다 훨씬 느슨함)
실행 판정 폭	주기 구간의 첫 60초(pollWindowSeconds) 안에만 실행 트리거 — 60초 폴링 주기와 맞물려 간격을 1분 미만으로 짧게 잡아도 매 폴링마다 1회씩만 실행됨
요일 판정	<code>*</code> (매일), 단일 숫자(0=일~6=토), 범위(<code>1-5</code> =평일) — collect와 동일 문법
중복 실행 방지	배치는 "오늘 실행했는지", 주기는 "이번 간격 구간(runIndex)에 실행했는지"를 서버ID+파일경로+스케줄값 조합 키로 기록
재로딩 주기	스케줄 파일은 최대 60초(<code>precheck.analyze.scheduler.reload-interval-ms</code>) 캐싱, 수정 후 재기동 불필요

04 증분 분석 기준

INCREMENTAL ANALYSIS BASIS

TB_ANALYZE_HISTORY.lastAnalyzeLogId

"주기" 분석은 이 값 이후로 신규 적재된 `TB_COLLECT_LOG` 행만 조회해 분석합니다. collect의 `LAST_LINE_NUMBER` (파일 내 라인번호)와는 별개 개념으로, **COLLECT_LOG_ID(PK) 기준**의 진행 위치입니다. 분석 시작 직전 `STATUS=FAIL`, `FAIL_REASON=IN_PROGRESS` 로 이력을 선등록하므로, JVM이 비정상 종료해도 "어디까지 처리했는지"가 DB에 남습니다(재기동 시 자동 재처리는 안 되며 수동 확인 필요).

05 작성 체크리스트 · 주의사항

CHECKLIST & COMMON MISTAKES

- ✓ 서버ID는 collect 스케줄 파일의 서버ID와 **정확히 동일한 문자열**이어야 함(다르면 해당 서버 로그가 전혀 분석되지 않음)
- ✓ 새 서버를 collect에 추가했다면 analyze 스케줄에도 반드시 등록 — collect만 등록하면 로그는 쌓이는데 분석·판정은 되지 않음
- ✓ 주기 분석의 간격(분)을 지나치게 짧게 잡아도 실제로는 60초 폴링 주기에 종속되므로 1분 미만으로 촘촘히 잡을 실익이 적음

✓ 시작시각 ≥ 종료시각인 라인은 조용히 무효 처리되니, 스케줄 등록 후 로그([[주기 분석 실행 결과]])로 실제 트리거 여부를 확인할 것

collect에만 서버 등록

수집은 되지만

PreCheck_AnalyzeLogs_Schedule.conf 에 서버ID가 없으면 TB_COLLECT_LOG 에 로그만 쌓이고 TB_ANALYZE_RESULT 는 전혀 생성되지 않습니다.

같은 서버ID를 중복 등록하며 값이 다름

마지막 라인이 최종 적용되므로, 실수로 위쪽에 다른 스케줄을 남겨두면 "설정했는데 반영이 안 된다"고 오인하기 쉽습니다. 중복 등록 여부를 먼저 확인하세요.

파일경로 표기 불일치를 문제로 착각

analyze는 파일을 직접 읽지 않으므로 파일경로 값 자체는 분석 결과에 영향을 주지 않습니다. 분석이 안 될 때는 파일경로가 아니라 서버ID 일치 여부를 먼저 점검하세요.

배치와 주기를 혼동

배치는 하루 1회, 주기는 반복 실행입니다. 실시간에 가까운 감시가 필요한 서버에 배치를 쓰면 하루 종일 지연 발생 시에도 다음 배치 시각까지 알 수 없습니다.